浙江大学

本科实验报告

课程名称：　电子电路设计实验 II

姓　　名：

学　　院：　信息与电子工程学院

系：

专　　业：　信息工程

学　　号：

指导教师：　施红军、叶险峰、邓靖靖

2026 年 4 月 23 日

专业：信息工程
姓名：
学号：
日期：2026/4/23
地点：

浙江大学实验报告

课程名称：电子电路设计实验 II　指导老师：施红军、叶险峰、邓靖靖　成绩：
实验名称：基本实验测试实验报告　同组学生姓名：

一、实验目的

1. 学习使用 Arduino 的多引脚数字输出控制，通过红、绿、黄三色 LED 模拟交通信号灯的依次切换，掌握函数封装与串口状态反馈的编程方法，并理解延时参数对时间控制的作用。
2. 掌握 Arduino 模拟引脚的信号采集方法，学习使用串口通信实时输出传感器数据，理解模拟数值与物理电压之间的线性换算关系。
3. 熟悉 Arduino 引脚的数字输出控制方法，掌握循环与顺序结构实现多引脚分时操作的编程技巧，通过多组 LED 组合的交替点亮与熄灭，理解动态扫描显示的基本原理，并观察延时时间长短对视觉暂留效果的影响。

二、实验任务与要求

1. 实验一

基于 Arduino 的数字信号输出实验：数字交通灯

2. 实验二

基于 Arduino 的模拟信号输出实验：可变电阻

3. 实验三

基于 Arduino 的特殊信号输出实验：PWM LED 呼吸灯实验

要求基于所给的代码块或已完成的完整代码完成 Arduino 代码的编写，并进行硬件搭建，将 Arduino 板连接至个人 PC 上传代码，并实现完整功能，利用串口监视器观察输出结果。

三、实验原理与步骤

1 实验原理

1.1. 实验一：数字交通灯

Arduino 上的数字接口既可以作为输入，也可以作为输出，但是不能同时作为输入和输出。为了明确特定数字接口的输入/输出方式，需要使用 pinMode 函数:

pinMode (pin, MODE)；

参数 pin 明确数字接口的编号。参数 MODE 确定输入或输出模式，该参数有三种取值：

INPUT, 常规输入模式

INPUT_PULLUP, 内部上拉电阻的输入模式

OUTPUT, 输出模式

Arduino 对每个数字接口提供了激活内部上拉电阻的选项, INPUT_PULLUP 模式就是用来指定是否启用。数字接口的内部上拉电阻。

对于输出模式，代码中可以设定该数字接口的输出电平为 HIGH 或 LOW。具体使用 digitalWrite 函数:

digitalWrite(pin, value)；

参数 pin 指定目标数字接口的编号。参数 value 设定指定数字接口的输出电压为 HIGH 或 LOW。
对于小电流设备，如 LED ，必须串联电阻（通常为 1kΩ ）进行限流。
对于大电流设备，需要在数字接口外接一个继电器或 BJT 进行电流放大。

1.2. 实验二：可变电阻

Arduino 共有 6 个模拟信号输入针脚。
为了能够读取模拟信号，Arduino 提供模/数转换器(ADC)。ADC 将模拟信号转换为数字信号。由于转换后的数字信号与输入的模拟信号有对应关系，根据转换后的数字信号的数值就可以确定原始模拟信号的大小。
数字信号中，Arduino 通过每一个固定长度的 0 和 1 的变化组合成字节，再解码成文字信息。而模拟信号中，靠的是分辨率。分辨率指的是 Arduino 或某个设备可以感知到的细节。
电路中通常说的几位的分辨率，这个“位”是 2 的指数。Arduino 是 10 位的分辨率，可以在其能容忍的最大电压值中分辨出 1024（2^{10}）种变化。Arduino 的标准最大测量电压为 5V，所以每一刻度的数值是：

$$\frac{5\text{V}}{1024} = 0.0049\text{V}$$

一般来说，模拟信号测量的单位就是电压，单位为 V。

1.3. 实验三：PWM LED 呼吸灯实验

脉冲宽度调制，简单来说就是利用数字输出的引脚来模拟模拟信号的输出，因为不需要通过额外的数模转换过程，而且数字信号抗干扰的能力也比较强，因此，PWM 在很多设备中用作控制信号。
不同型号的 Arduino 支持 PWM 的针脚数量并不相同，以目前主流的 Arduino UNO 来说，全部共有 6 个针脚，也就是 3、5、6、9、10、11。
要 RGB 引脚输入不同的电压，就可以分别产生颜色深浅变化。可采用 PWM，实现 RGB LED 经由不同的电压变化来达成颜色的转换。

装 订 线

2 实验方案设计

2.1. 实验一：数字交通灯

实现代码如下：
本段代码逻辑通过定义红、黄、绿三种状态对应的持续时间（分别为 6 秒、2 秒、6 秒），在 setup 函数中将数字引脚 10、11、12 配置为输出模式，分别对应绿灯、黄灯和红灯。在 loop 主循环中，程序依次调用 stoplight（红灯亮）、golight（绿灯亮）和 yieldlight（黄灯亮）函数，从而形成一个完整的交通灯循环周期。每个函数内部通过 digitalWrite 控制引脚的高低电平来点亮或熄灭 LED，并利用 delay 函数维持亮灯时长，同时通过串口打印当前状态供调试使用。

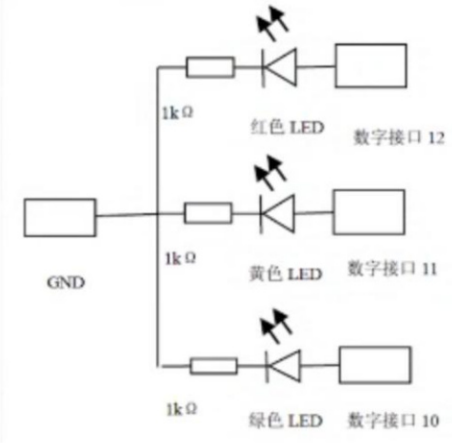
```
int red = 6;
int yellow = 2;
int green = 6;

void setup() {
    Serial.begin(9600);
    pinMode(10, OUTPUT);
    pinMode(11, OUTPUT);
    pinMode(12, OUTPUT);
    digitalWrite(10, LOW);
    digitalWrite(11, LOW);
    digitalWrite(12, LOW);
}
void golight(int time) {
    digitalWrite(10, HIGH);
    digitalWrite(11, LOW);
    digitalWrite(12, LOW);
    Serial.print("Light mode: Go - ");
    Serial.println(time);
    delay(time * 1000);
}
void loop() {
    stoplight(red);
    golight(green);
    yieldlight(yellow);
}

void stoplight(int time) {
    digitalWrite(10, LOW);
    digitalWrite(11, LOW);
    digitalWrite(12, HIGH);
    Serial.println("Light mode: Stop");
    delay(time * 1000);
}

void yieldlight(int time) {
    digitalWrite(10, LOW);
    digitalWrite(11, HIGH);
    digitalWrite(12, LOW);
    Serial.println("Light mode: Yield");
    delay(time * 1000);
}
```

接线方案如下：



装

订

线

2.2. 实验二：可变电阻

实现代码如下：

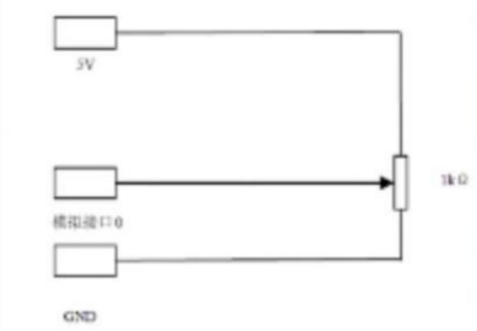
```
int analogPin = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int reading = analogRead(analogPin);
  float voltage = reading / 204.6;
  Serial.print("Reading=");
  Serial.print(reading);
  Serial.print("\t Volts=");
  Serial.println(voltage);
  delay(1000);
}
```

本段代码用于读取模拟传感器的数据并将其转换为电压值。代码将模拟引脚 0 定义为输入源，在 setup 中初始化串口通信波特率为 9600。在 loop 循环中，程序使用 analogRead 读取引脚的模拟值（范围 0-1023），随后通过公式“读数×0.0049”将其转换为 0-5V 范围内的电压值

（基于 5V 参考电压和 10 位分辨率）。最后，程序通过串口每秒打印一次原始读数和计算后的电压值。这种程序常用于连接电位器、光敏电阻或 LM35 温度传感器等模拟输出设备。
接线方案如下：



2.3. 实验三：PWM LED 呼吸灯实验

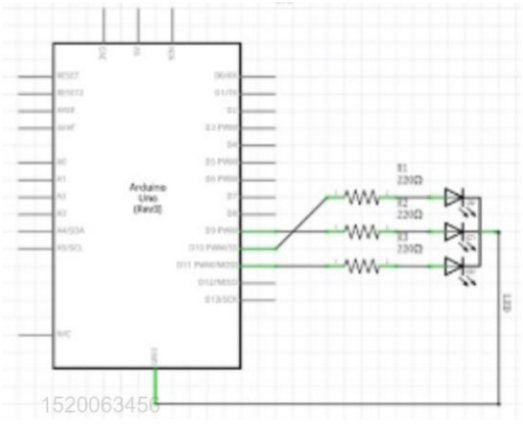
实验代码如下：

```
void setup() {
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
}

void loop() {
  // 红色呼吸（通道9）
  breathe(9);
  // 绿色呼吸（通道10）
  breathe(10);
  // 蓝色呼吸（通道11）
  breathe(11);
}

// 单通道呼吸函数（与第一段代码逻辑一致）
void breathe(int pin) {
  for (int i = 0; i < 255; i++) {
    analogWrite(pin, i);
    delay(10);
  }
  delay(50);
  for (int i = 255; i > 0; i--) {
    analogWrite(pin, i);
    delay(10);
  }
  delay(50);
}
```

这段代码实现了 RGB LED 的三色呼吸灯效果。在 setup 中，程序将引脚 9、10、11 配置为输出，这三个引脚通常连接 RGB LED 的红、绿、黄三个通道。代码核心在于自定义的 breathe 函数，该函数利用 analogWrite 输出 PWM 信号来模拟电压变化。通过两个 for 循环，函数分别控制引脚的 PWM 占空比从 0 线性增加到 255（渐亮）以及从 255 线性减小到 0（渐暗），中间配合 delay 延时以控制呼吸速度。在 loop 主循环中，程序依次对三个引脚调用该函数，从而产生红色、绿色、黄色 LED 依次进行呼吸闪烁的视觉效果。
接线方案如下：



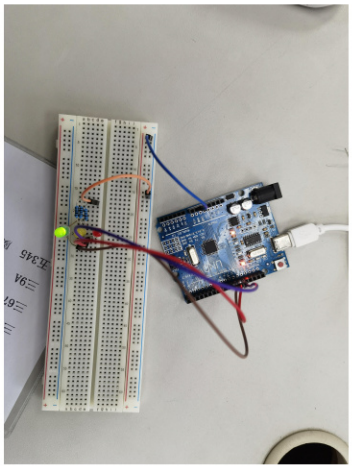
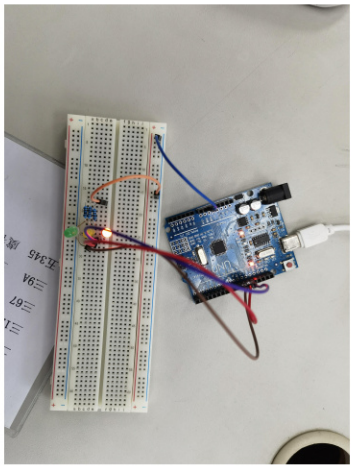
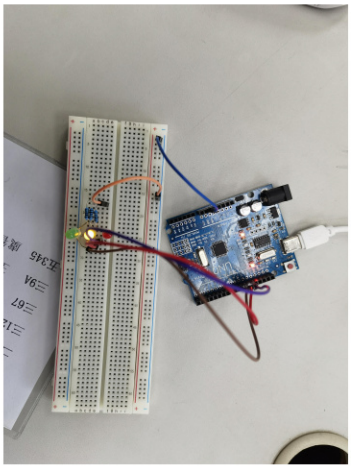
装 订 线

四、实验结果记录

1　实验一：数字交通灯

按接线方案，接好电路，并将代码通过 Arduino IDE 代码上传至 Arduino UNO 板，可执行对应功能。

交通灯接好后效果如下图所示。三灯依次亮起，红、黄、绿三种状态对应的持续时间分别为 6 秒、2 秒、6 秒，如下图所示。

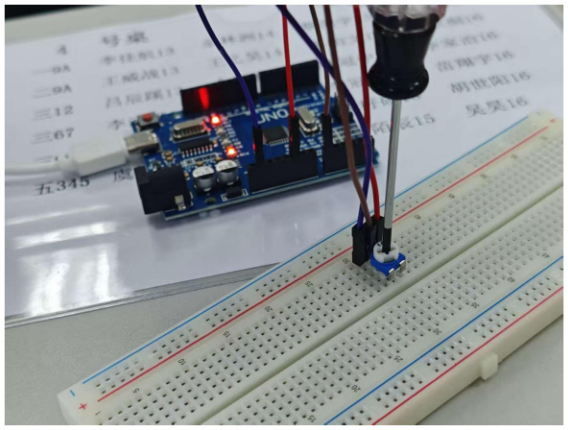
串口监视器结果如下图所示：



2　实验二：可变电阻

按接线方案，接好电路，并将代码通过 Arduino IDE 代码上传至 Arduino UNO 板，可执行对应功能。

接线方式如下：



使用螺丝刀扭动可变电阻改变两端阻值，电压大小将会反映两端的阻值变化，串口监视器输出如下图所示。

```
Reading=0       Volts=0.00
Reading=1       Volts=0.00
Reading=99      Volts=0.48
Reading=180     Volts=0.88
Reading=248     Volts=1.21
Reading=320     Volts=1.56
Reading=406     Volts=1.98
Reading=517     Volts=2.53
Reading=598     Volts=2.92
Reading=645     Volts=3.15
Reading=761     Volts=3.72
Reading=870     Volts=4.25
Reading=1023    Volts=5.00
```

如图所示，分辨率为 0～1023，每刻度对应 0.0049V。

3　实验三：PWM LED 呼吸灯实验

按接线方案，接好电路，并将代码通过 Arduino IDE 代码上传至 Arduino UNO 板，可执行对应功能。

代码如下：

```
void setup() {
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);
}

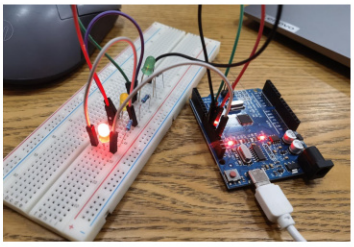
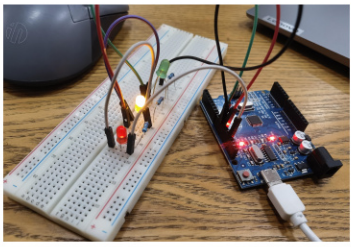
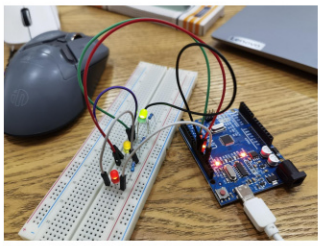
void loop() {
  // 红色呼吸（通道9）
  breathe(9);
  // 绿色呼吸（通道10）
  breathe(10);
  // 蓝色呼吸（通道11）
  breathe(11);
}

// 单通道呼吸函数（与第一段代码逻辑一致）
void breathe(int pin) {
  for (int i = 0; i < 255; i++) {
    analogWrite(pin, i);
    delay(10);
  }
  delay(50);
  for (int i = 255; i > 0; i--) {
    analogWrite(pin, i);
    delay(10);
  }
  delay(50);
}
```

这段代码实现了 RGB LED 的三色呼吸灯效果。在 setup 中，程序将引脚 9、10、11 配置为输出，这三个引脚通常连接 RGB LED 的红、绿、黄三个通道。代码核心在于自定义的 breathe 函数，该函数利用 analogWrite 输出 PWM 信号来模拟电压变化。通过两个 for 循环，函数分别控制引脚的 PWM 占空比从 0 线性增加到 255（渐亮）以及从 255 线性减小到 0（渐暗），中间配合 delay 延时以控制呼吸速度。在 loop 主循环中，程序依次对三个引脚调用该函数，从而产生红色、绿色、黄色 LED 依次进行呼吸闪烁的视觉效果。

接线方式及最终效果如下：

根据观察，每个灯的呼吸时间大约为 10 秒。

五、心得与体会

实验一的数字交通灯主要使用了 pinMode、digitalWrite、delay 和 Serial 函数。pinMode(pin, OUTPUT) 将数字引脚配置为输出模式，决定了引脚可以向外驱动电平；digitalWrite(pin, HIGH/LOW) 则精确控制 LED 的亮灭。实际编写时，需要合理安排 delay 的长短并配合 Serial.print 在串口监视器中输出状态信息。将红灯、绿灯、黄灯分别封装成 stoplight、golight、yieldlight 三个函数后，loop 主循环变得非常清晰，这就是代码可读性和可维护性的重要性。

实验二通过电位器读取模拟电压，核心函数是 analogRead 和 Serial.print。analogRead(A0) 返回 0～1023 的整数值，对应 Arduino UNO 内置 10 位 ADC 对 0～5V 电压的量化结果。在代码中，可使用 reading/204.6 将读数换算成电压，这其实是一个简化的线性转换。当转动电位器时，串口监视器上原始读数和电压值同步变化，这就是“分辨率”的含义，每变化一个数字量，电压就改变约 0.0049V。analogRead 并不需要事先用 pinMode 设置引脚模式，因为模拟引脚默认就是输入，这一点和数字引脚的习惯有所不同。

实验三的 RGB LED 呼吸灯主要侧重于 analogWrite 函数的用法。analogWrite(pin, value) 并不是输出真正的模拟电压，而是输出 PWM 方波，value 从 0 到 255 控制占空比，通过占空比的改变，就可以有效驱动方波电压的有效值变化，进而改变 LED 的亮度。利用 PWM 模拟电压变化既简单又高效，而且不占用额外的数模转换电路。在两个 for 循环中，亮度值由小到大再到大到小，LED 便呈现出平滑的呼吸效果。在代码中将这个渐亮渐暗的过程封装成 breathe 函数，依次传入三种颜色对应的引脚，即可实现红、绿、黄灯的轮流呼吸，体现的是 C++当中的程序模块化思想，将需要反复使用的功能单独作为一个模块编写可以很好的提升代码可读性。

本次实验同时锻炼了代码编写能力与软硬件调试能力。